

Scottish Hills Topographical Elevation Study

Spatial distribution profiling, Munro filtering, elevation differences modeling, and regional terrain analysis.

Dataset: scottish_hills.csv (282 Peaks)

Attributes: Height (m), Region, Lat/Long

Tech: Pandas, NumPy, Matplotlib

1. Scenario Breakdown & Geographical Outcomes

Scenario 1: Data Loading & Basic Cleaning

Loaded 282 mountain records. Derived 2-letter Ordnance Survey regional prefixes (NN, NH, NG, etc.). Imputed missing elevation records with mean ($\mu = 1,003.5\text{m}$) and region with mode ('NN').

Scenario 2: Sequential Sample Elevation Profiling

Extracted heights of the first 10 hills into NumPy arrays to evaluate local variance. Heights range from 918.0m to 1,120.0m. Line graph: hill_heights_line.png.

Scenario 3: Munro Filtering (Height > 900m)

Identified 281 peaks meeting Munro criteria (>900m). Region **NN (Grampian: 140 peaks)** and **NH (North: 69 peaks)** encompass over 74% of all tall peaks. Bar chart: tall_hills_bar.png.

Scenario 4: Top 5 Region Distribution Share

Quantified regional density: NN (49.6%), NH (24.5%), NG (11.0%), NO (8.5%), and NM (2.8%). Pie chart: region_distribution.png.

Scenario 5: Advanced Stratification & NumPy Differentials

Categorized elevations: *Very High* ($\geq 1000\text{m}$: 133 peaks, 47.2%), *High* (800–999m: 148 peaks, 52.5%), *Moderate* (<800m: 1 peak). Computed sequential differences with `np.diff()`. Tallest peak: **Ben Nevis (1,344.5m)**. Visuals: height_trend.png, height_category_stacked.png, height_histogram.png.

2. Visual Representation

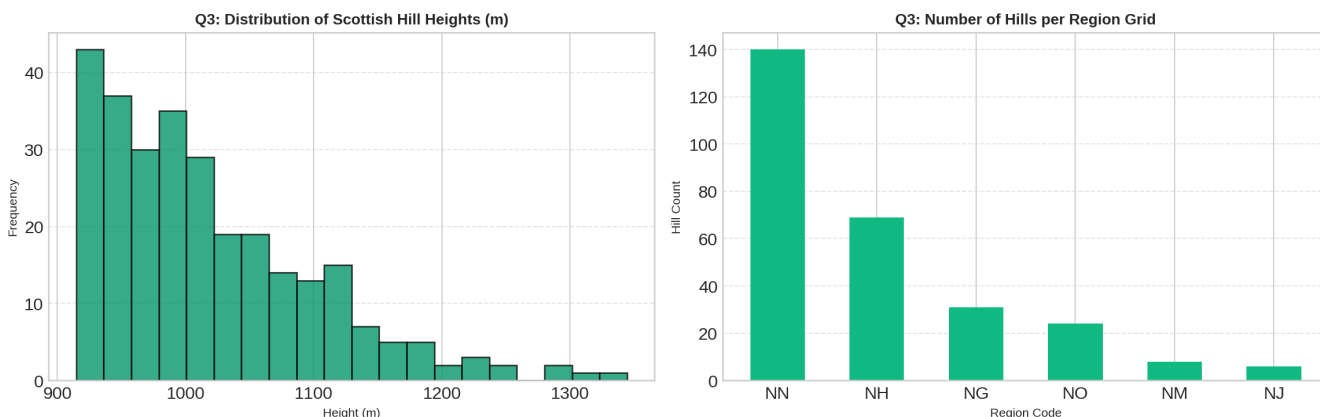


Figure 3.1: Elevation Distribution Histogram (Left) & Regional Mountain Counts (Right).

3. Python Code Snippet

```
import os, pandas as pd, numpy as np, matplotlib.pyplot as plt
df = pd.read_csv("scottish_hills.csv")
if "Region" not in df.columns: df["Region"] = df["Osgrid"].astype(str).str[:2]
df["Height"] = pd.to_numeric(df["Height"], errors='coerce').fillna(df["Height"].mean())
diffs = np.diff(df["Height"].to_numpy())
print(f"Tallest Peak: {df.loc[df['Height'].idxmax(), 'Hill Name']} ({df['Height'].max()}m)")
```